

Practical - 2

Government College of Engineering, Jalgaon (An Autonomous Institute of Govt. of Maharashtra) Department of Computer Engineering	
Name:	
PRN:	Batch:
Class:	Semester:
Course:	Subject Teacher:
Date of Performance:	Date of Completion:

Aim : Program for Distributed Application using RPC

Theory :

What is RPC?

Remote Procedure Call (RPC) is an interprocess communication technique. The Full form of RPC is Remote Procedure Call. It is used for client-server applications. RPC mechanisms are used when a computer program causes a procedure or subroutine to execute in a different address space, which is coded as a normal procedure call without the programmer specifically coding the details for the remote interaction.

This procedure call also manages low-level transport protocol, such as User Datagram Protocol, Transmission Control Protocol/Internet Protocol etc. It is used for carrying the message data between programs.

Callback RPC

This type of RPC enables a P2P paradigm between participating processes. It helps a process to be both client and server services.

Functions of Callback RPC:

- Remotely processed interactive application problems
- Offers server with clients handle
- Callback makes the client process wait
- Manage callback deadlocks
- It facilitates a peer-to-Peer paradigm among participating processes.

Broadcast RPC

Broadcast RPC is a client's request, that is broadcast on the network, processed by all servers which have the method for processing that request.

Functions of Broadcast RPC:

- Allows you to specify that the client's request message has to be broadcasted.
- You can declare broadcast ports.
- It helps to reduce the load on the physical network

Batch-mode RPC

Batch-mode RPC helps to queue, separate RPC requests, in a transmission buffer, on the client-side, and then send them on a network in one batch to the server.

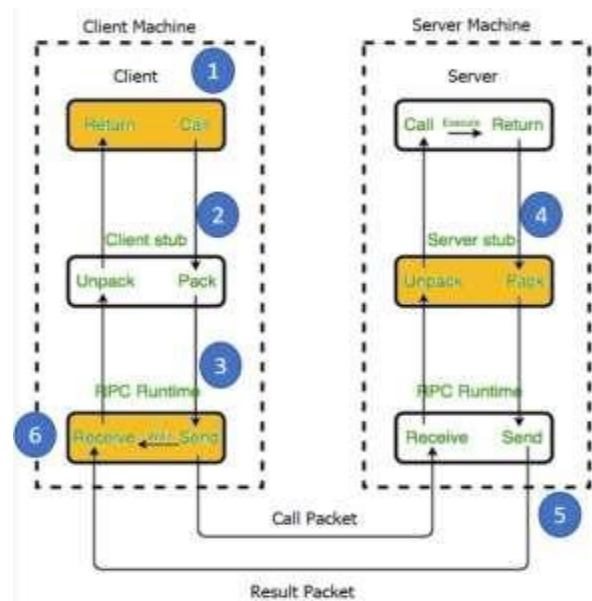
Functions of Batch-mode RPC:

- It minimizes overhead involved in sending a request as it sends them over the network in one batch to the server.
- This type of RPC protocol is only efficient for the application that needs lower call rates.
- It needs a reliable transmission protocol.

RPC Architecture

RPC architecture has mainly five components of the program:

1. Client
2. Client Stub
3. RPC Runtime
4. Server Stub
5. Server



RPC Architecture

How RPC Works?

Following steps take place during the RPC process:

Step 1) The client, the client stub, and one instance of RPC run time execute on the client machine.

Step 2) A client starts a client stub process by passing parameters in the usual way. The client stub stores within the client's own address space. It also asks the local RPC Runtime to send back to the server stub.

Step 3) In this stage, RPC accessed by the user by making regular Local Procedural Cal. RPC Runtime manages the transmission of messages between the network across client and server. It also performs the job of retransmission, acknowledgment, routing, and encryption.

Step 4) After completing the server procedure, it returns to the server stub, which packs (marshalls) the return values into a message. The server stub then sends a message back to the transport layer.

Step 5) In this step, the transport layer sends back the result message to the client transport layer, which returns back a message to the client stub.

Step 6) In this stage, the client stub demarshalls (unpack) the return parameters, in the resulting packet, and the execution process returns to the caller.

Characteristics of RPC

Here are the essential characteristics of RPC:

- The called procedure is in another process, which is likely to reside in another machine.
- The processes do not share address space.
- Parameters are passed only by values.
- RPC executes within the environment of the server process.
- It doesn't offer access to the calling procedure's environment.

Features of RPC

Here are the important features of RPC:

- Simple call syntax
- Offers known semantics
- Provide a well-defined interface
- It can communicate between processes on the same or different machines

Code :

Server.py

```
from xmlrpc.server import SimpleXMLRPCServer

def reverse_str(s1):
    ch = list(s1)
    rev = ""

    for i in range(len(ch) - 1, -1, -1):
        rev += ch[i]

    return rev

server = SimpleXMLRPCServer(("localhost", 1030))

print("Initializing Server...")
print("Server Ready on port 1030")

server.register_function(reverse_str, "input")

server.serve_forever()
```

Client.py

```
import xmlrpc.client
proxy = xmlrpc.client.ServerProxy("http://localhost:1030/")
name3 = ""

while True:
    print("\n\t\t*** Menu ***")
    print("\n 1. Send Input String")
    print("\n 2. Display Reversed String")
    print("\n 0. Exit")
    ch = int(input("\nEnter your choice: "))
```

```
if ch == 1:
    name1 = input("Enter First String: ")
    name3 = proxy.input(name1)

elif ch == 2:
    print("\nReversed String =", name3)

elif ch == 0:
    break

else:
    print("Invalid choice")
```

Output :

Client

```
D:\Me\College\DSL\2_DistributedApp_MPI>python server.py
Initializing Server...
Server Ready on port 1030
127.0.0.1 - - [08/Apr/2026 23:05:26] "POST / HTTP/1.1" 200 -
```

Server

```
D:\Me\College\DSL\2_DistributedApp_MPI>python client.py

*** Menu ***

1. Send Input String
2. Display Reversed String
0. Exit

Enter your choice: 1
Enter First String: Distributed Systems

*** Menu ***

1. Send Input String
2. Display Reversed String
0. Exit

Enter your choice: 2

Reversed String = smetsyS detubirtsid
```

Conclusion : In this practical we learnt about RPC and its implementation.

Sign of Teacher
Mrs. Archana Chitte.